

keda-gpu-scaler: GPU-aware autoscaling for Kubernetes from real hardware metrics

Pavan Madduri¹

¹ Independent Researcher, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 21 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

keda-gpu-scaler is a Kubernetes External Scaler for KEDA ([KEDA Authors, 2024](#)) that drives horizontal autoscaling of GPU workloads from GPU hardware metrics. It runs as a DaemonSet on GPU nodes, reads per-device state directly from the NVIDIA Management Library (NVML) ([NVIDIA Corporation, 2024c](#)), and serves those metrics to KEDA over the external scaler gRPC contract. Workloads can then scale on GPU utilization, memory use, or engine-level signals, including scaling to and from zero replicas. Beyond raw device metrics, the scaler can read serving-engine signals such as vLLM ([Kwon et al., 2023](#)) queue depth and NVIDIA Triton queue statistics, so scaling can track request pressure rather than only device saturation. It supports Multi-Instance GPU (MIG) partitions and several aggregation strategies (max, min, average, sum, and percentiles) for nodes with multiple GPUs.

Statement of need

The Kubernetes Horizontal Pod Autoscaler reacts to CPU and memory, neither of which reflects GPU saturation. A GPU inference pod can sit at single-digit CPU utilization while its GPU is fully saturated, so CPU- and memory-based autoscaling either over-provisions or fails to respond. The common workaround chains dcgm-exporter into Prometheus, evaluates PromQL, and feeds the result back through KEDA. That pipeline adds several moving parts and tens of seconds of metric latency, which is significant when GPU capacity is expensive and demand is bursty.

keda-gpu-scaler removes that pipeline. Because it reads NVML on the node and answers KEDA directly, scaling decisions are based on live hardware state without an intermediate metrics store or query language. This matters for large language model serving and other GPU inference workloads, where operators need to scale on GPU utilization or on serving-engine backpressure, and where scale-to-zero meaningfully reduces cost. The project targets platform and machine learning infrastructure engineers who run GPU workloads on Kubernetes and want autoscaling that reflects what the accelerator is doing.

State of the field

GPU metric-driven autoscaling in Kubernetes is typically achieved through a multi-stage pipeline. NVIDIA's dcgm-exporter ([NVIDIA Corporation, 2024a](#)) scrapes GPU counters into Prometheus, PromQL queries are evaluated by a Prometheus adapter or by KEDA's Prometheus scaler, and the result drives the Horizontal Pod Autoscaler. The NVIDIA GPU Operator ([NVIDIA Corporation, 2024b](#)) automates driver and device-plugin lifecycle but delegates scaling to this same Prometheus path. GKE's custom-metrics-stackdriver-adapter and similar managed-cloud solutions exist but are vendor-locked and unavailable on bare-metal or multi-cloud clusters.

keda-gpu-scaler was built as a new project rather than contributed to these existing tools for architectural reasons. KEDA's external-scaler gRPC contract is the only extension point that allows a node-local daemon to push live hardware state directly into the scaling loop without an intermediate time-series database. Embedding GPU logic into KEDA core would couple KEDA's release cycle to NVML and vendor-specific dependencies; KEDA's documentation directs metric sources without a built-in scaler to the external scaler interface (KEDA Authors, 2026). Contributing to dcgm-exporter would not eliminate the Prometheus dependency, which is the primary source of latency and operational complexity that this project removes.

Software design

keda-gpu-scaler runs as a Kubernetes DaemonSet: one pod per GPU node, with direct access to the NVIDIA Management Library through go-nvml CGo bindings. This node-local architecture was chosen over a centralized controller because GPU state is per-host and NVML requires local device access.

The core abstraction is a Collector interface that decouples metric acquisition from the gRPC serving layer. The default implementation calls NVML; a vendor-agnostic factory (contributed by an external developer) selects the appropriate collector at startup, enabling future support for AMD ROCm or Intel Level Zero without changing the server. A mock collector enables full test coverage without GPU hardware.

The gRPC server implements KEDA's four-method ExternalScaler contract. Scaling decisions are configured entirely through ScaledObject trigger metadata. No custom resources or operator are required. Aggregation strategy (max, average, percentile) is a runtime parameter, allowing operators to tune fleet-wide scaling behavior without code changes.

Serving-engine integration (vLLM, Triton) is implemented as composable metric sources that layer on top of device metrics, so scaling can combine hardware saturation with request-queue pressure in a single ScaledObject.

Research impact statement

The design was described in a CNCF blog post on building a GPU external scaler for KEDA (Madduri, 2026a) and covered in articles in VKTR (Madduri, 2026b) and Techstrong.ai (Madduri, 2026c), and the project is discussed in an IEEE Communications Society Technology Blog article on scaling agentic AI in telecom networks (Madduri & Thakur, 2026). It is the subject of an accepted session at KubeCon + CloudNativeCon North America 2026 (Cloud Native Computing Foundation, 2026).

At the time of writing the repository has 130 stars, 36 forks, and merged pull requests from thirteen external contributors, including a vendor-agnostic GPU collector factory (PR #222) and a Terraform stack for reproducible GPU cluster provisioning with Terratest end-to-end tests. There are eight tagged releases between May and September 2026 and 248 non-merge commits, with CI on every pull request, Dependabot, OpenSSF Scorecard, and CodeQL enabled.

Functionality

The scaler exposes the four methods of the KEDA external scaler interface and is configured entirely through standard ScaledObject trigger metadata. Key capabilities include:

- Per-device metrics from NVML: utilization, memory used and total, temperature, and power draw.
- Serving-engine metrics: vLLM pending queue depth and KV-cache usage, and Triton queue statistics, for request-aware scaling.

- Multi-GPU aggregation strategies (max, min, avg, sum, p95, p99).
- MIG per-instance metrics for partitioned GPUs.
- Scale-to-zero and configurable cooldown to prevent scale-down flapping.
- An optional Prometheus endpoint for fleet monitoring, independent of the scaling path.

The design and rationale, including why GPU support is implemented as a standalone external scaler rather than embedded in KEDA core, are documented in the repository.

AI usage disclosure

GitHub Copilot was used for code-completion assistance during development, and Claude (Anthropic) was used to assist with drafting documentation and this paper. All code, design decisions, architectural choices, and paper content were reviewed, validated, and edited by the human author. The core design, a DaemonSet with direct NVML access behind KEDA's external-scaler gRPC contract, was conceived and implemented by the author based on experience operating GPU inference workloads on Kubernetes.

Acknowledgements

The project builds on the KEDA project (KEDA Authors, 2024), the NVIDIA Management Library (NVIDIA Corporation, 2024c), and the broader Kubernetes ecosystem (Burns et al., 2016).

References

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1), 70–93. <https://doi.org/10.1145/2898442.2898444>
- Cloud Native Computing Foundation. (2026). *KubeCon + CloudNativeCon North America 2026: Speaker page, Pavan Madduri*. <https://kubcon-cloudnativecon-north-america-2026.sessionize.com/speaker/d15f6f95-f72b-4f32-8206-c04856f67ea0>.
- KEDA Authors. (2024). *KEDA: Kubernetes event-driven autoscaling*. <https://keda.sh>.
- KEDA Authors. (2026). *KEDA external scalers*. <https://keda.sh/docs/latest/concepts/external-scalers/>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. <https://doi.org/10.1145/3600006.3613165>
- Madduri, P. (2026a). *GPU autoscaling on Kubernetes with KEDA: Building an external scaler*. <https://www.cncf.io/blog/2026/05/27/gpu-autoscaling-on-kubernetes-with-keda-building-an-external-scaler/>.
- Madduri, P. (2026b). *Why Kubernetes can't see your GPUs, and what we built to fix it*. <https://www.vktr.com/ai-technology/why-kubernetes-cant-see-your-gpus-and-what-we-built-to-fix-it/>.
- Madduri, P. (2026c). *Your AI agent can't see your GPUs, here's how to fix that*. <https://techstrong.ai/features/your-ai-agent-cant-see-your-gpus-heres-how-to-fix-that/>.
- Madduri, P., & Thakur, A. L. (2026). *The financial trap of autonomous networks: Scaling agentic AI in the telecom core*. <https://techblog.comsoc.org/2026/03/30/the-financial-trap-of-autonomous-networks-scaling-agentic-ai-in-the-telecom-core/>.

- 125 NVIDIA Corporation. (2024a). *NVIDIA DCGM Exporter*. [https://github.com/NVIDIA/dcgm-](https://github.com/NVIDIA/dcgm-exporter)
126 [exporter](https://github.com/NVIDIA/dcgm-exporter).
- 127 NVIDIA Corporation. (2024b). *NVIDIA GPU Operator*. [https://github.com/NVIDIA/gpu-](https://github.com/NVIDIA/gpu-operator)
128 [operator](https://github.com/NVIDIA/gpu-operator).
- 129 NVIDIA Corporation. (2024c). *NVIDIA Management Library (NVML)*. [https://developer.](https://developer.nvidia.com/management-library-nvml)
130 [nvidia.com/management-library-nvml](https://developer.nvidia.com/management-library-nvml).

DRAFT